

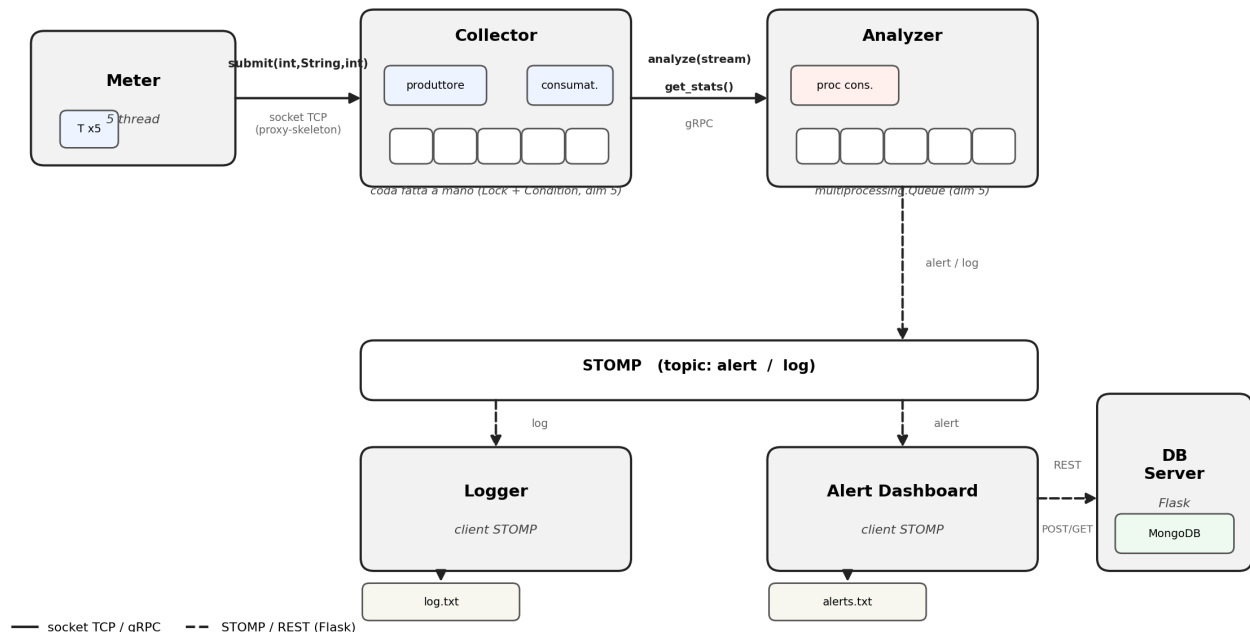
Esame di Advanced Computer Programming

Prova pratica di esercitazione — *Mega-prova integrativa*

Generata da intelligenza artificiale

Lo studente legga attentamente il testo e produca il programma ed i casi di test necessari per dimostrarne il funzionamento. La mancata compilazione dell'elaborato, la compilazione con errori o l'esecuzione errata daranno luogo alla valutazione come prova non superata. Al termine della prova lo studente dovrà far verificare il funzionamento del programma ad un membro della Commissione.

Testo della prova



Il candidato implementi un sistema distribuito in **Python** per il monitoraggio dei consumi energetici e la gestione di allarmi, basato su **Socket TCP**, **gRPC**, **STOMP**, **Flask** e **MongoDB**. Il sistema è caratterizzato dai seguenti componenti.

Meter. È un client multithread utilizzato per l'invio di letture di consumo verso il **Collector**. L'invio di una lettura consiste nell'invocazione del metodo `void submit(int, String, int)` specificato nell'interfaccia **ICollector**. La richiesta è caratterizzata da 1) **meterId** (`int`), identificativo univoco del contatore; 2) **zone** (`String`), la zona di appartenenza, scelta casualmente tra *north* e *south*; 3) **reading** (`int`), il valore di consumo, generato casualmente nel range [50, 150]. Il Meter crea 5 thread: ogni thread invoca 4 volte il metodo `submit` (attendendo 1 secondo tra le invocazioni). I valori di meterId, zone e reading sono generati come descritto. La comunicazione tra Meter e Collector deve avvenire tramite proxy-skeleton su socket TCP; il candidato predisponga a tal fine l'interfaccia **ICollector** e le opportune classi Proxy e Skeleton.

Collector. Fornisce l'interfaccia **ICollector** ed il relativo metodo `void submit(int, String, int)`. Il metodo `submit`, che deve essere eseguito in mutua esclusione, avvia un produttore che inserisce in una coda una stringa che concatena i parametri ricevuti, e.g., *north-12-87*. La coda deve essere implementata "a mano" (NON utilizzando le code della libreria standard), thread-safe, di dimensione 5, e deve risolvere il problema del produttore/consumatore utilizzando i costrutti di sincronizzazione **Lock** e **Condition**. All'avvio del Collector viene lanciato un consumatore che preleva le stringhe dalla coda. Il Collector funge inoltre da **client gRPC** verso l'Analyzer:

- il consumatore accumula le letture prelevate dalla coda; ogni 5 letture apre uno stream gRPC ed invia le 5 letture all'Analyzer invocando il metodo `string analyze(stream Reading)` — interazione di tipo **gRPC streaming client** — e stampa a video la stringa di risposta restituita;

- dopo l’invio di ciascun batch, il consumatore invoca il metodo *Stats get_stats()* — interazione di tipo **unary** — e stampa a video le statistiche ricevute (numero di letture analizzate e valore medio dei consumi).

Si utilizzi inoltre skeleton per ereditarietà per il Collector.

Analyzer. È un **server gRPC** e **client STOMP** che riceve le richieste dal Collector ed espone i metodi *analyze* e *get_stats*. All’avvio, l’Analyzer crea un processo consumatore (modulo *multiprocessing*) che preleva i valori da una **Queue** del modulo *multiprocessing* (process-safe, di dimensione 5) e li analizza: per ogni valore che supera una determinata soglia (fornita da terminale), crea un *frame STOMP* nel quale inserisce la stringa *zone-meterId-reading* e lo invia sul topic **alert**; in caso contrario (valore minore o uguale alla soglia) lo invia sul topic **log**. Il processo consumatore aggiorna inoltre dei contatori condivisi (numero di letture e somma dei valori) necessari al calcolo delle statistiche.

- il metodo *analyze* (client-streaming) riceve lo stream di letture dal Collector e, per ciascuna lettura ricevuta, la inserisce nella Queue *multiprocessing*; al termine dello stream ritorna al chiamante la stringa *OK*;
- il metodo *get_stats* (unary) ritorna al chiamante il numero di letture analizzate ed il valore medio dei consumi (attenzione ai potenziali problemi di concorrenza nell’accesso ai contatori condivisi).

Alert Dashboard. È un client STOMP in attesa asincrona sul topic **alert**. Alla ricezione di un nuovo *frame STOMP*, la dashboard avvia un nuovo processo che: 1) stampa a video la lettura ricevuta; 2) scrive in append sul file *alerts.txt* il contenuto del messaggio; 3) effettua una richiesta **HTTP POST** verso il **DB Server**, inserendo nel body, in formato json, i campi *zone*, *meterId* e *reading* estratti dal messaggio, e.g., {“zone”:”north”, “meterId”:12, “reading”:140}.

Logger. È un client STOMP in attesa asincrona sul topic **log**. Alla ricezione di un *frame STOMP*, estrae il contenuto del messaggio e lo scrive in append sul file *log.txt* (oltre a stamparlo a video).

DB Server. Implementa un server **Flask** che gestisce una collection **MongoDB** contenente gli allarmi ricevuti, ed espone una **REST API** con due endpoint:

- un endpoint per la registrazione di un allarme che, ricevuta una richiesta con payload json (descritto in precedenza), crea un *document* all’interno della collection MongoDB contenente tutti i campi ricevuti;
- un endpoint che, ricevuta una zona, restituisce il numero di allarmi registrati per quella zona ed il valore medio dei relativi consumi, calcolati sui *document* presenti nella collection.

Lo studente progetti la REST API, scegliendo opportunamente i metodi HTTP da utilizzare.

Lo studente dovrà sviluppare tutti i componenti. Il sistema sarà testato da terminale, in quest’ordine di avvio, con: 1 DB Server, 1 Logger, 1 Alert Dashboard, 1 Analyzer (con la soglia passata da riga di comando), 1 Collector e 1 Meter.